Transformer is All You Need

1. Data Preparation and Tokenization

The Tiny Shakespeare corpus is a single text file of about 1.1 million characters. Before any modeling, we need to turn these huge numbers of characters into a sequence of integer tokens the Transformer can embed and predict over. I implemented Byte-Pair Encoding (BPE) from scratch. The idea behind BPE is quite simple. Start with every unique character in the corpus as a base token for Tiny Shakespeare that gives 65 tokens, mostly letters, punctuation, and whitespace. Then repeatedly look at every pair of adjacent tokens in the corpus, find the pair that occurs most often, and merge it into a new token. Each merge adds one entry to the vocabulary and shortens the token stream. We stop once the vocabulary reaches the target size (500) or no pair appears more than once.

My implementation keeps two dictionaries (id_to_str and str_to_id) and a list of learned merges in the order they were applied. For training, I encode the whole corpus into a list of integer ids, count adjacent pairs with a single counter pass, pick the most frequent one, and rewrite the id stream to replace every occurrence of that pair with a fresh id. This repeats until the vocabulary is full. At encoding time, I replay the merges in the exact order they were learned, which is important, applying merges in a different order would produce a different tokenization.

With vocab_size = 500, BPE learned 435 merges on top of the 65 base characters. The merges it picked line up well with intuition about English: common short sequences like th, the, ou, an showed up early. A few Shakespeare-specific patterns also appeared, like :\n and \n\n, which

reflect the play's formatting (a speaker name followed by a newline, blank lines between stanzas). This is a nice sanity check that BPE is adapting to the statistics of this particular corpus rather than some generic English prior.

I verified the tokenizer with a round-trip test: encode a 120-character snippet, decode the resulting ids, and check that the output matches the input exactly. It does, which confirms that encoding and decoding are consistent. The same snippet compressed from 120 characters down to 55 tokens, roughly 2.2 characters per token. That ratio matters for training: a higher characters-per-token means each sequence of fixed length covers more raw text, so the model sees longer range context for the same compute budget. One limitation worth noting is that the tokenizer is trained on the full corpus before the train/val split. Strictly speaking, this leaks some statistics from the validation portion into the vocabulary.

2. Sequence Formatting

After tokenization, the corpus is a list of 506296 integer ids. The Transformer expects fixed-length sequences, so the next step is to slice this stream into short windows that can be batched.

For next-token prediction, each example is a pair of sequences: the input is a window of N consecutive tokens, and the target is the same window shifted forward by one position. The model produces one prediction per position, and cross-entropy loss compares each prediction to

its target. A single 64-token window therefore gives the model 64 next-token prediction problems to learn from, not just one, which is a big reason Transformers train efficiently.

I picked a context length of 64 tokens. This is short by modern LLM standards but appropriate here, since the model is tiny and attention cost scales up with sequence length. Rather than chopping the corpus into disjoint windows, I slide a window across the stream with a stride of 32, giving 50% overlap between consecutive windows. A stride equal to the window length would give cleaner examples but only about 7900 windows; a stride of 1 would produce near-duplicates. A stride of 32 sits in the middle and yields 15820 training windows.

For the split, I used torch.utils.data.random_split with a fixed generator seed for reproducibility, which gives 12656 training windows and 3164 validation windows. One thing worth flagging: because adjacent windows overlap, a validation window can share up to 32 tokens with a nearby training window. A stricter setup would split the corpus into contiguous halves before building windows. I kept the random split because it matches the assignment's instructions and the validation loss still behaves sensibly as a training signal. Inputs and targets are wrapped in a small NextTokenDataset and fed through PyTorch DataLoaders with batch size 64.

3. Tiny Transformer Architecture

The model is a small decoder only Transformer in the style of GPT. It has 2 blocks, **d_model = 128**, 4 attention heads, **ffn_dim = 256**, and dropout 0.1, for a total of 328576 parameters.

Token ids first pass through an nn.Embedding lookup that maps each of the 500 vocabulary entries to a 128-dimensional vector. Self-attention is permutation-invariant on its own, so I add a fixed sinusoidal positional encoding from "Attention Is All You Need" to inject position information at no parameter cost.

Each Transformer block follows a pre-norm layout: x = x + Attention(RMSNorm(x)), then x = x + FeedForward(RMSNorm(x)). Pre-norm is more stable to train than the original post norm formulation and is what modern LLMs use. RMSNorm rescales features by their root-mean-square but skips the mean-centering step of LayerNorm, which makes it slightly cheaper. The attention block projects the input into Query, Key, and Value with one fused linear layer, splits each into 4 heads of dimension 32, and computes scaled dot-product attention $\text{softmax}(QK^T / \sqrt{d_k})\, V$. Before the softmax, a precomputed upper triangular mask sets all future positions to -inf so they receive zero attention weight. This causal mask is what makes next-token prediction work; without it, position 5 could just peek at position 6. The feed forward network is a position-wise MLP that expands 128 to 256, applies GELU, and projects back to 128. The residual connections around both sublayers let gradients flow and let each block learn small refinements on top of identity.

After the final block, a last RMSNorm is applied and a linear layer projects to 500 vocabulary logits. I used weight tying between the input embedding and the output projection, sharing the same matrix to cut parameters and slightly improve perplexity.

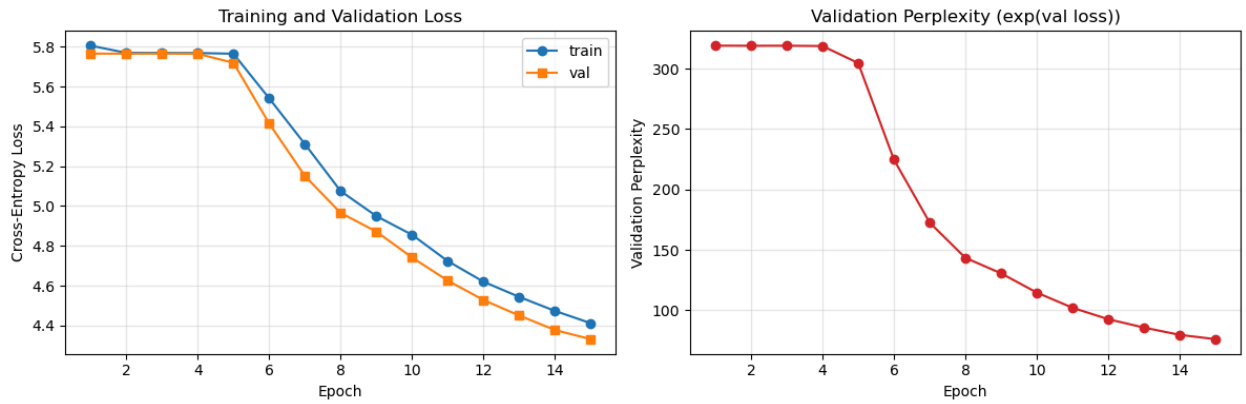
4. Training and Results

I trained the model with cross-entropy loss using AdamW (learning rate 3e-4, weight decay 0.01) for 15 epochs with batch size 64. Gradients are clipped to a max norm of 1.0 to keep training stable. Validation perplexity is the main metric, since it tells us roughly how many tokens the model is choosing between at each position.

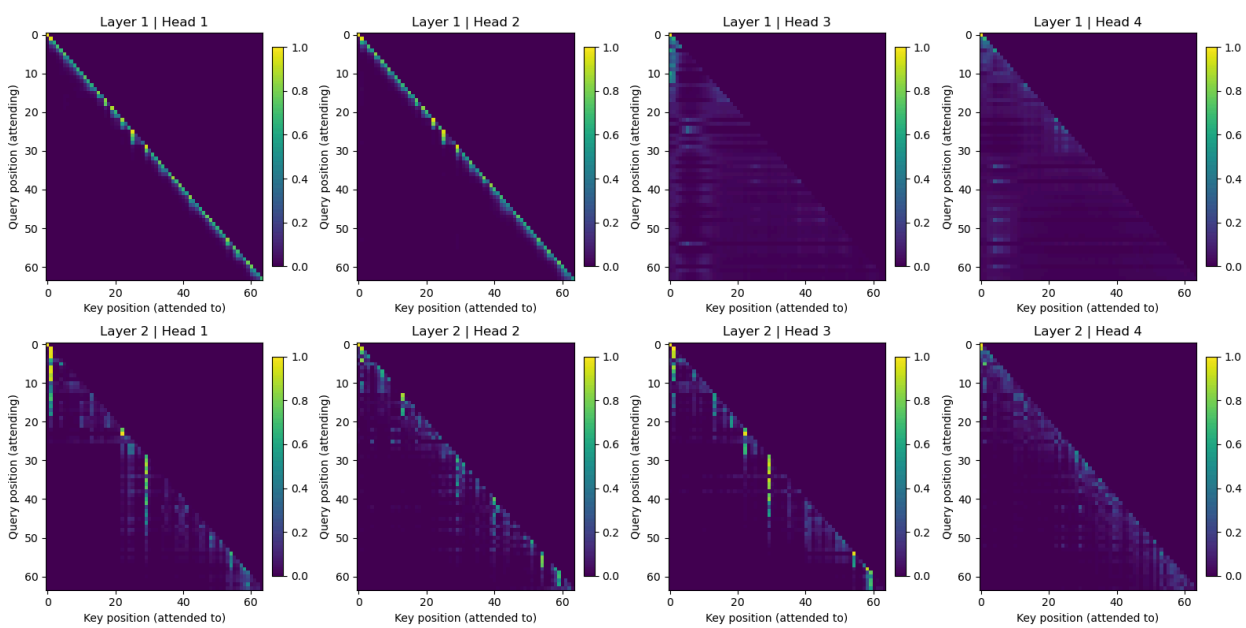
Training split into two clear phases. For the first four to five epochs the loss barely moved, hovering near the uniform-guess baseline of about 319 perplexity. The model was mostly learning which tokens are common during this stretch. Around epoch 5 to 6 things clicked and loss started dropping quickly, which is a common pattern for small Transformers: attention needs a few epochs to become useful before the rest of the network can take advantage of it.

By the end of training, validation loss reached 4.34 and validation perplexity reached 76.81, roughly a 4x improvement over the uniform baseline. Train and validation curves stayed close throughout, with no signs of overfitting, suggesting there would be room to train longer or scale the model up. For a network of this size on a 1MB corpus, this is a reasonable result and confirms the architecture is working at the end.

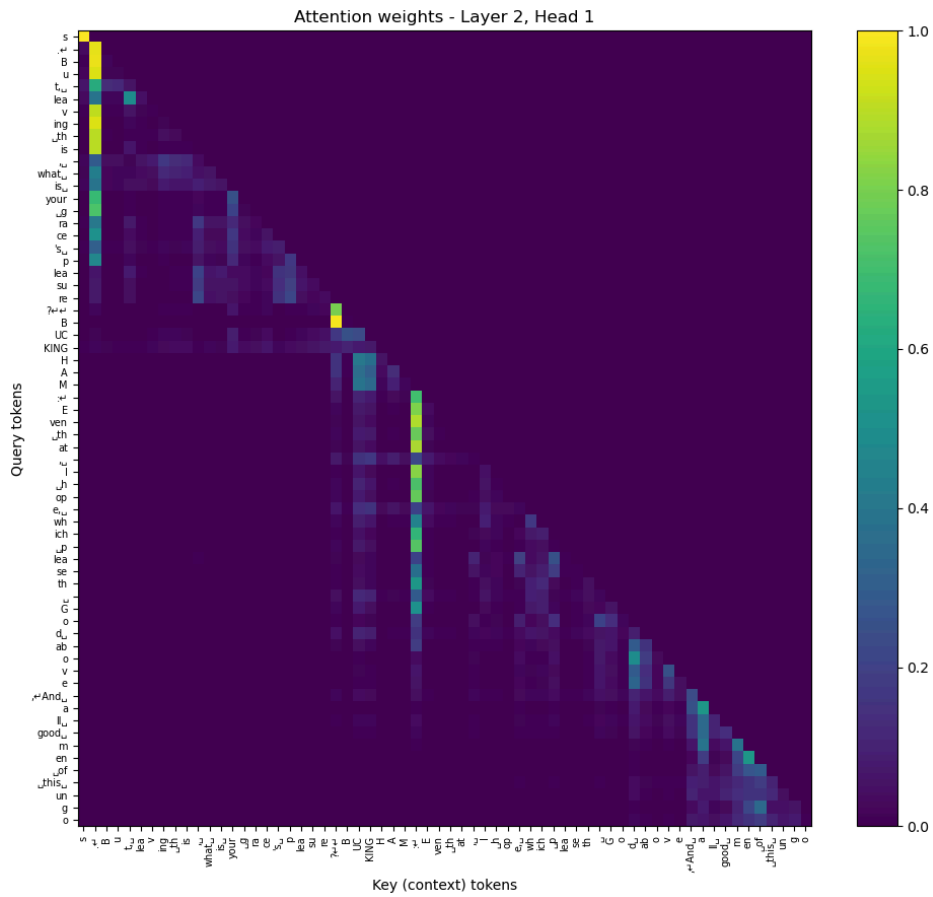
5. Visualization & Interpretation



In this visualization, both curves stay flat for the first four epochs near 5.77 cross-entropy and 319 perplexity, then drop sharply starting at epoch 5. The validation curve is slightly below the training curve throughout, which is the expected effect of dropout being on at training time and off at evaluation. There is no gap opening up between the two, so the model is not overfitting and would likely benefit from more epochs or a larger size.



Here are the per-head attention heatmaps. The four heads in each layer learn clearly different jobs. In layer 1, heads 1 and 2 lock onto a sharp diagonal: each query attends mostly to itself and the immediately preceding token, picking up local character and short-subword patterns. Heads 3 and 4 are much more diffuse, spreading weight broadly across the past with a noticeable bright leftmost column, behaving more like global context aggregators. Layer 2 looks structurally different: the patterns are sharper and several heads show vertical stripes where many query positions all attend to the same earlier key position.



Then, looking at Layer 2 Head 1 with token labels makes the structural patterns concrete. Two strong vertical stripes dominate: one on the very first token (the attention sink behavior, where queries with no clear target default to position 0) and one around the speaker tag tokens like B,

UC, KING. Many later query tokens attend back to that speaker name, suggesting the head has learned that the speaker label is a useful anchor for predicting what comes next in the play's structure. There is also a faint diagonal band, meaning the head still keeps some weight on the immediately preceding token.

6. Discussion

I agree with what professor Luo mentioned during the lecture, the modern bottleneck of the transformer/attention mechanism is when the scale of data goes up, the run time goes up exponentially (O(n^2)). At a context of 64 this is fine, but doubling to 128 would roughly quadruple per step compute. Memory was not a constraint at this scale: the whole model fits well on MPS and a full epoch runs in under a minute. The real bottleneck is data, not computation. Moreover, in terms of the role of position encoding, self-attention is permutation-invariant on its own, so without positional information the model would treat a sequence and any shuffle of it identically. Removing the sinusoidal encoding would mean the model could not tell whether a token came at the start or end of the input, which is fatal for next-token prediction.

7. AI Tool Usage Disclosure

I used the Huggingface BPE tokenizer example to build the BPE class first. I asked ChatGPT to help me debug during the class. Then, I used Claude to guide me through how to build the Tiny Transformer step by step and ask it to give me explicit explanations on each line of code it

generates. I tried my best to understand the logic between each line of code. Finally, I used Gemini to help me make the visualization look neat and clean.